Lecture 1

08/08/2023 (1)

Automata theory → Theory of computation

↓

theoretical branch of Computer Science and Mathematics

↓

deals with logic of computation w.r.t simple machines

↓

Automata

Automata enables scientists to understand how machines compute the functions and solve problems.

Importance

↓

vital role in problem solving by providing systematic approach.

It helps in breaking down complex problems into smaller and more manageable components.

By applying theoretical concepts, computer scientists can efficiently design algorithms that solve specific issues.

Ex String = abc

Substring → a, b, c, ab, bc, abc, ε → 6

$$\frac{n(n+1)}{2} + 1 = \frac{3 \times 4}{2} + 1 = 7$$

Total no. of substring including ε

$$= \boxed{\frac{n(n+1)}{2} + 1}$$

without ε

$$\boxed{\frac{n(n+1)}{2}}$$

Q String = abcd

Substring → a, b, c, d,
ab, bc, cd,
abc, bcd,
abcd

10 + ε = 11

$$\frac{4(5)}{2} + 1 = 11$$

* K length substrings

$n - k + 1$

abc

a, b, c, | ✓

ab, bc, |

abc

$n = 3$

K = length substring

= 2

$3 - 2 + 1 = 2$

ab, bc → (2)

* for n length of string,

n+1 no. of suffixes

Toc →

E, c, oc, Toc

↓

3

(3)

Transition Diagram

Transition table

Present State	Next State for i/p 0	Next State i/p 1
→ q_0	q_0	q_1
q_1	q_2	q_1
* q_2	q_2	q_2

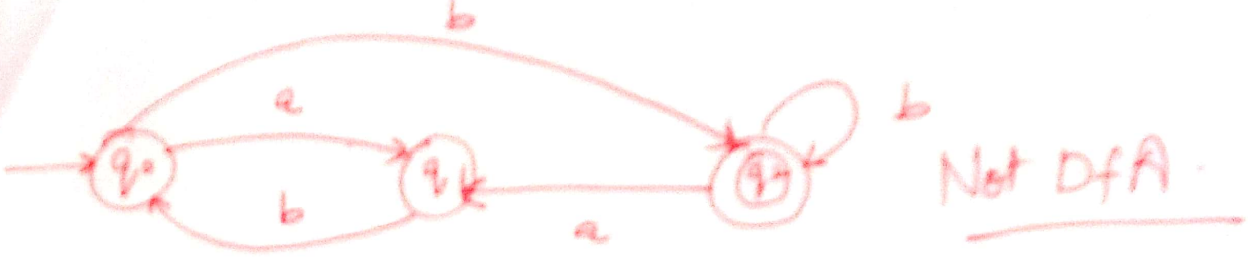
* Convert Table to diagram and vice versa

* Acceptance of string in DfA

String Scan Left to Right
if DfA enters a final state

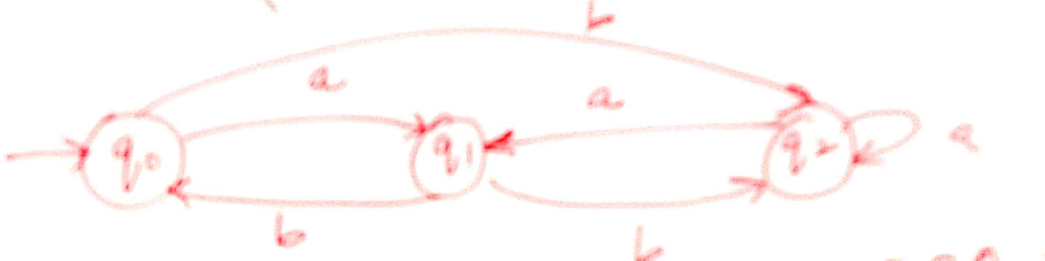
$a^* \; b\,b^*$

$a^n b^m \quad n \geq 0, m \geq 1$



Not DfA

$\delta(q_1, b) \rightarrow$ does not exist



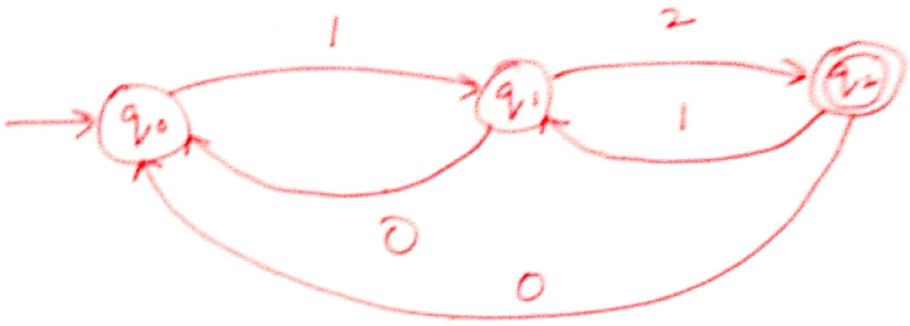
No final state Not DfA

Automata
Alphabet
String
Substring
Suffix / Prefix
Language
formal Language
FA
Types
DfA

* FA of 3

0
1
2

2 → 1 → 0
2 → 1
1 → 0



Language Recognizer FA

↓

is used in design of compiler.

O/P generator → is used in circuit design area.

LR → accepts valid string

DFA
NFA
ENFA

Automata

i/p ⟶ o/p

* formal language

Collection of strings where strings are formed based on some conditions.

$L_1 = \{a^n b^m \mid n, m \geq 1\}$

$L_2 = \{a^n b^n \mid n \geq 1\}$

$L_3 = \{a^n b^n c^n \mid n \geq 1\}$

- Alphabet
- String
- Subset
- Prefix
- Suffix

* finite automata

mathematical model of which contains finite no. of states and transitions defined between them.

FA Types.

- Language Recognizer (DfA, NfA, ε-NfA)
- o/p Genetor (Mealy machine, Moore machine)

Automata

↓

is a machine that takes set of i/p's and produces o/p by processing i/p automatically.

Formal definition : → 5 tuple machine

$(Q, \Sigma, \delta, q_0, f)$

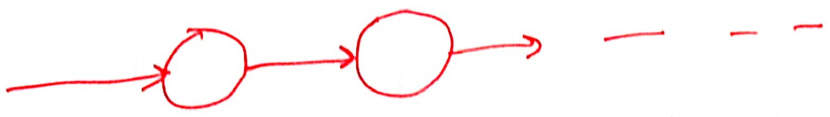
$Q \rightarrow$ finite set of states

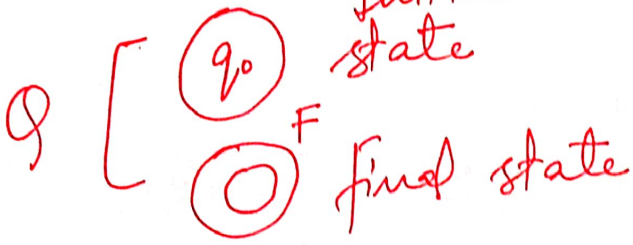
$\Sigma \rightarrow$ i/p alphabet $f \rightarrow$ set of final state

$q_0 \rightarrow$ initial state

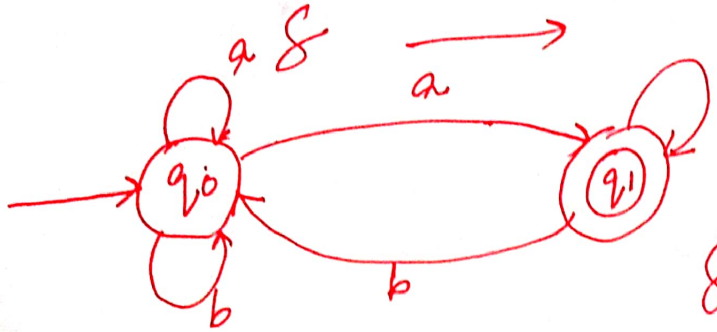
$\delta \rightarrow$ transition function

$\delta \rightarrow Q \times \Sigma \rightarrow Q$





$\{a, b, \ldots\} \{0, 1, \ldots\} \in$ alphabet



X

$\delta(q_0, a) \rightarrow q_0 / q_1$

X

Alphabet $\quad \varepsilon \quad 0 \quad 1$

$\{a, b, c\}$

$\{0, 1, \_\_\}$] → Strings

Any set of strings over a given alphabet.

$\Sigma = \{0, 1\}$

$L = \{0, 01, 11\}$ $\quad \{=\}$

→ $L = \{\}$ empty language.

$L = \{0, 1, 00, 01, 10, \ldots\}$ ←

$L = \{\varepsilon\}$ finite language.

ε → epsilon. → i/p nothing → [ ] → final state

↓

String of length 0, and it is accepted → reach final state

$L = \{\varepsilon, 0, 1, 00, 01, 10, 11, \ldots\}$

Complete language (Σ^*)

ϕ → string ×

↓

never reach final state.

More examples

See if it is a DfA?

q_0 ✓

q_2 ✓

$Q = \{q_0, q_1, q_2\}$

$\Sigma = \{0, 1\}$

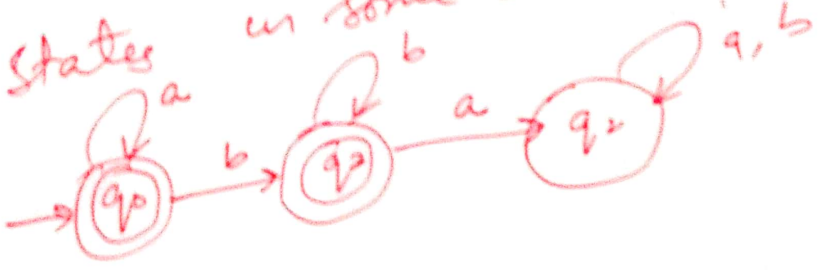
δ :

$\delta(q_0, 0) = q_0$
$\delta(q_0, 1) = q_1$
$\delta(q_1, 1) = q_1$
$\delta(q_1, 0) = q_2$
$\delta(q_2, 0) = q_2$
$\delta(q_2, 1) = q_2$

unique

DfA ✓

Final states in some examples => 1



DfA

It is a fA in which from each and every state on input symbol → exactly one transition should exist.

$(Q, \Sigma, \delta, q_0, F)$

$Q \rightarrow$ finite no. of states
$\Sigma \rightarrow$ I/p alphabets
$\delta \rightarrow Q \times \Sigma \rightarrow Q$ (Transition function)
$F \rightarrow$ set of final state/Accept state.
$q_0 \rightarrow$ Initial state.

Deterministics → uniqueness of computation

Graphical representation

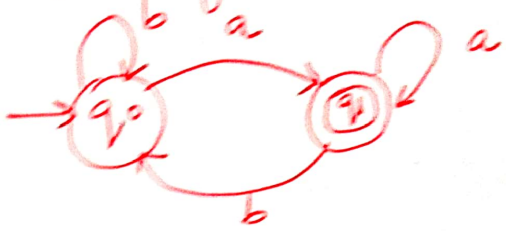

Initial state


Final state

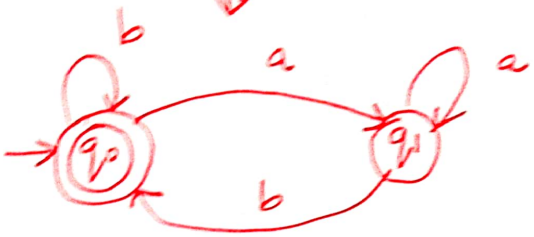

Trap state / dead state

* Complementing a DFA

Interchange final state and non-final states



Complement

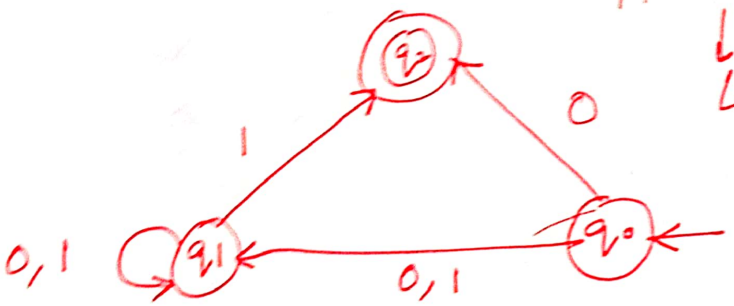


Automata

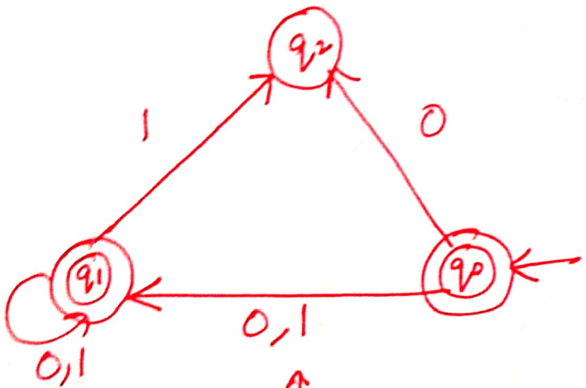
L ✓

~~DFA~~ ~~X~~

⇒



DfA X.



L_1 ✓

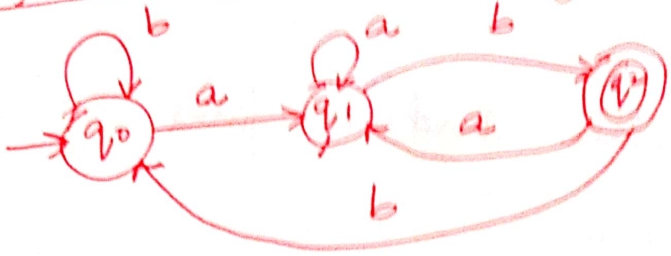
~~Not~~ ~~DFA~~

$\Sigma(0,1)^* \rightarrow \Sigma^*$

$\Sigma^0, \Sigma^1, \Sigma^2, \ldots$

* Questions: Acceptance of Strings in DfA

(1)



String:

a) Starts with ab
b) Ends with ab
c) Substring ab
d) None

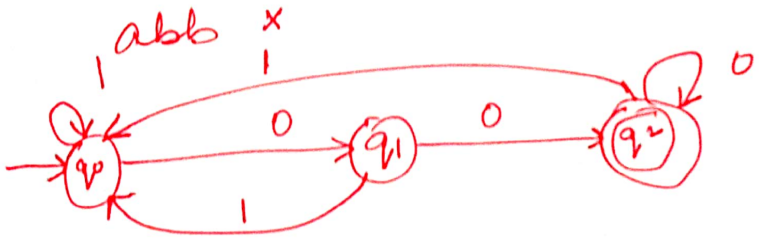
ba ✓ a) ×
ab ✓ b) ✓

Sub String → ab.

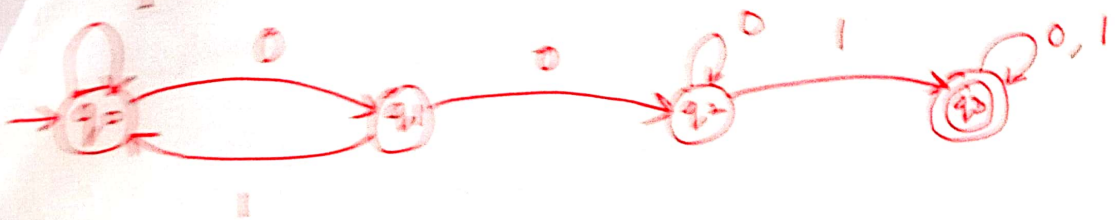
aba ×
abb ×

String accept
↓
ab.

(2)



a) ends with 0 ×
b) ends with 00 ×
c) Substring 00 × 0100.
d) ends with 00 ✓.



Language accepted :

(a) → strings starts with 001.

(b) → ends at 001.

(c) → substring 001.

(d) → none.

1001 ✗ (a)

0011 ✗ (b)

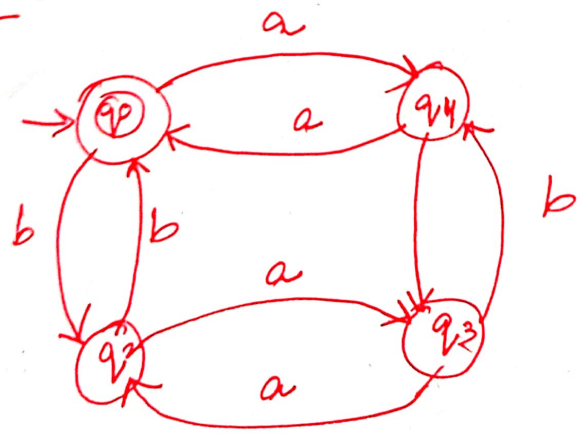
001 ✓ (c)

~~((0+1)*(0+1)*)*~~

$1^{*}\underline{0}\cdot(1+0)^{*}\cdot\underline{0}\cdot 0^{*}\cdot\underline{1}\cdot(0,1)^{*}$

001 ✓

(4) Identify language accepted by following DFA :-



✓ → even no. of a's and even no. of b's

→ even a's and odd b's

→ odd a's and even a's

→ odd a's and odd b's

Minimization of DFA :-

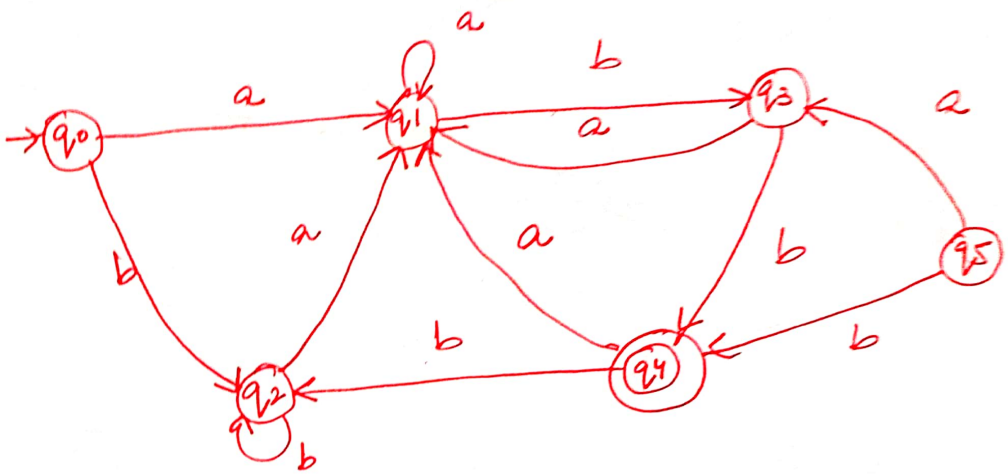
Minimization → possible for DFA.

(a) State Equivalence algorithm
X (b) Table filling algorithm X

(a) State Equivalence Algorithm :-

Two states are equivalent for all $x \in \Sigma^*$ both are going to either final or non-final state.

Ex :- 1



* (1) Inaccessible state should be removed.
(State not reachable from initial)

(2)

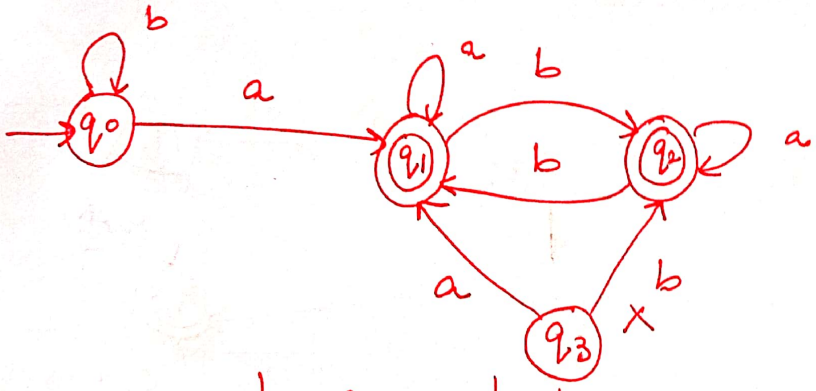
	a	b
→ q_0	q_1	q_2
q_1	q_1	q_3
q_2	q_1	q_2
q_3	q_1	q_4
* q_4	q_1	q_2

(3) $\{q_0, q_1, q_2, q_3\}$ $\{q_4\}$
Non-final final

$\equiv \{q_0, q_1, q_2\}, \{q_3\}, \{q_4\}$

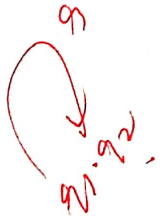
$\equiv \{q_0, q_2\}, \{q_1\}, \{q_3\}, \{q_4\}$

1:-



	a	b
→ q_0	q_1	q_0
* q_1	q_1	q_2
* q_2	q_2	q_1
q_3	q_1	q_2

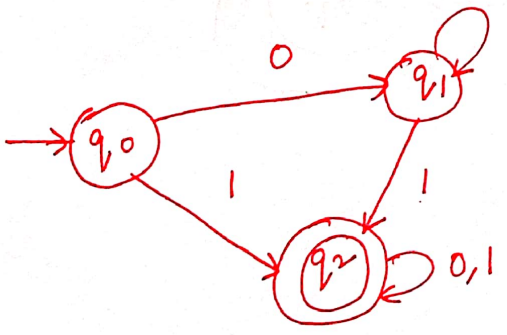
#



$\{q_0, q_3\}$ $\{q_1 q_2\}$

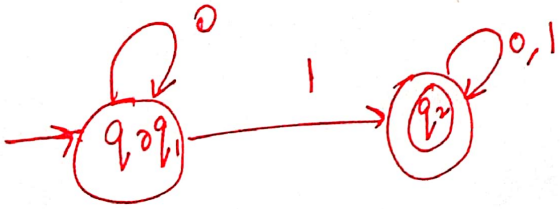
q_3 → eliminate

Ex 3:-

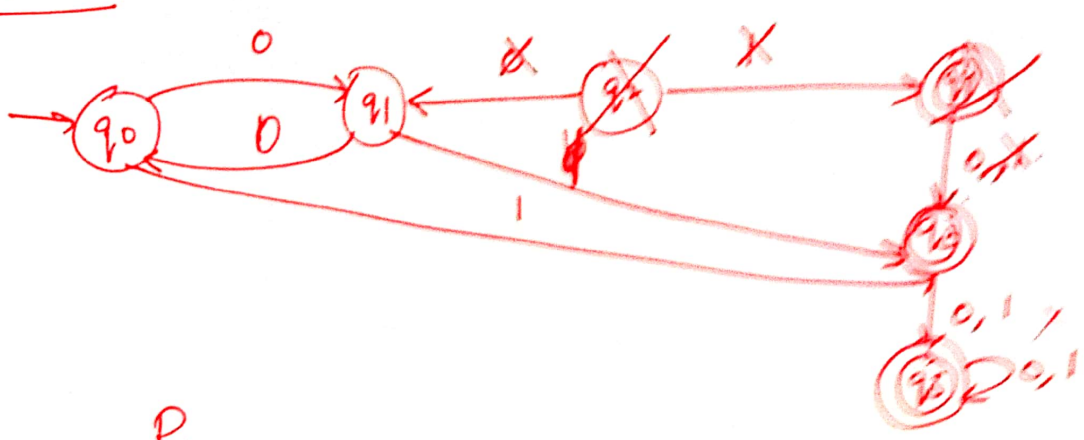


$\{q_0, q_1\}$ $\{q_2\}$

	a	b
→ q_0	q_1	q_2
q_1	q_1	q_2
* q_2	q_2	q_2



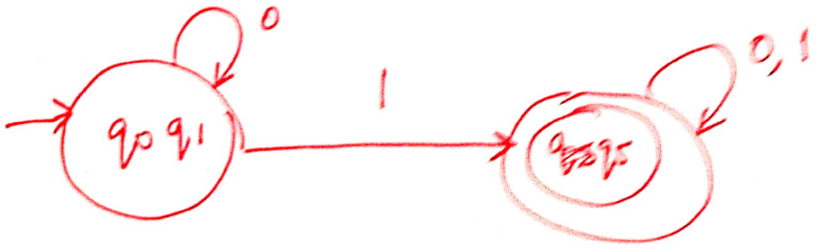
Ex 4:-

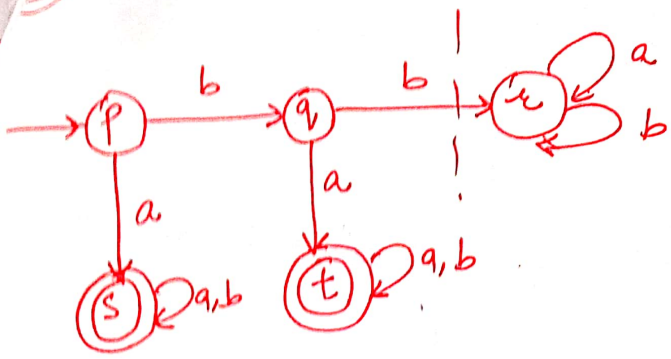


Remove q_2, q_4 as well

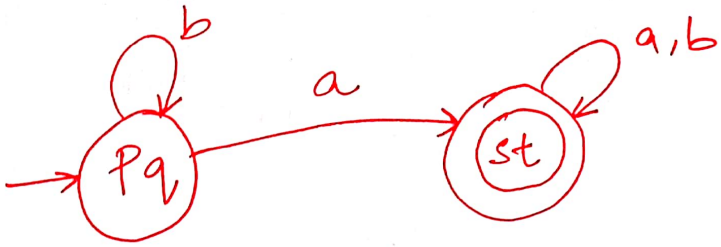
$\{q_0, q_1\}, \{q_3, q_5\}$

		0	1
→	q_0	q_1	q_3
	q_1	q_0	q_3
*	q_3	q_5	q_5
*	q_5	q_5	q_5





	a	b
→ p	s, t	q
q		
* s	s	s
* t	t	t



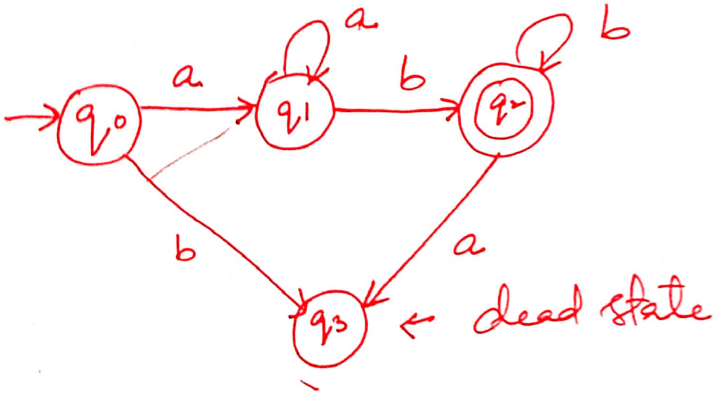
Construct DfA

Language contains any no. of a's followed by any no. of b's.

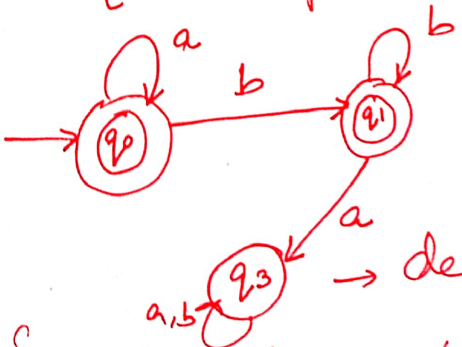
Set → $L = \{a^m b^n \mid m, n \geq 1\}$

(1) $= \{ab, aab, abb, aabb, aabbb, \cdots\}$

DfA → $(Q, \Sigma, \delta, q_0, f)$



(2) $L = \{a^n b^m \mid n, m \geq 0\}$



$\{\varepsilon, ab, a, b, aa, bb, abb, aab, \cdots\}$

(3) * $L = \{a^n b^m \mid n, m \geq 1, n \neq m\}$

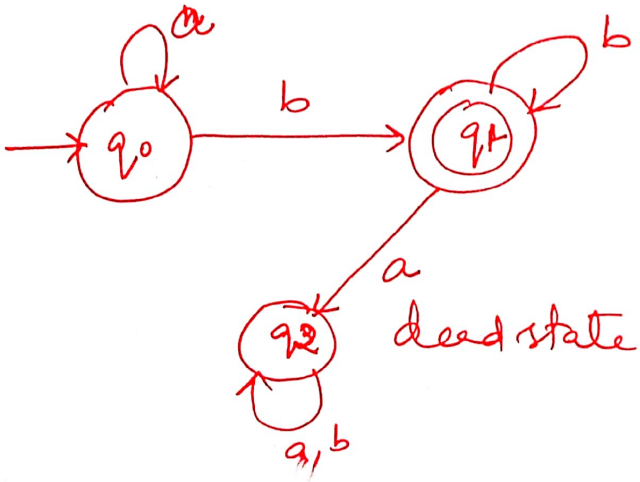
later

Not a regular language. ⟶ comparison.

Not accepted by DFA

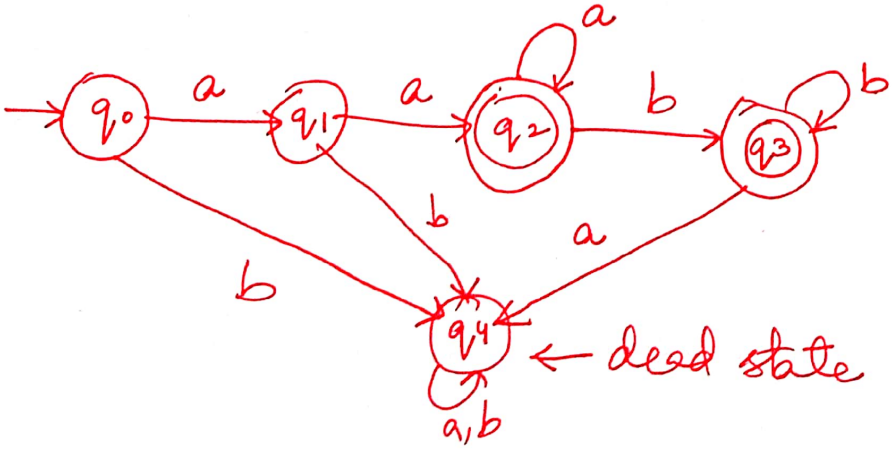
(4) $\{a^n b^m \mid n \geq 0, m \geq 1\}$

$L = \{b, bb, ab, aab, \cdots\}$



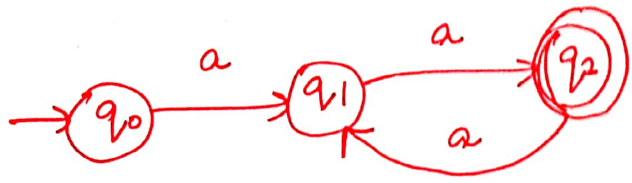
(5) $L = \{a^n b^m \mid n \geq 2, m \geq 0\}$

$L = \{aa, aaa, \ldots, aab, aabb, \ldots\}$



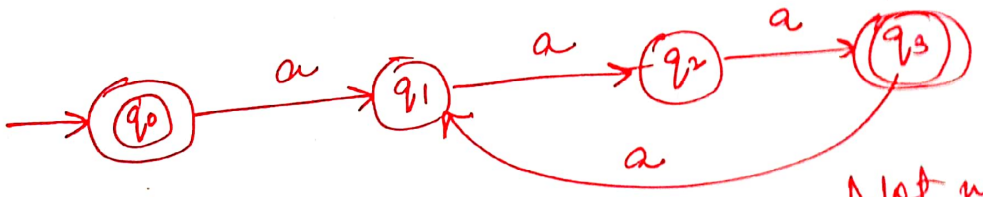
$L = \{a^{2n} \mid n \geq 1\}$

$= \{aa, aaaa, aaaaaa, \cdots\}$

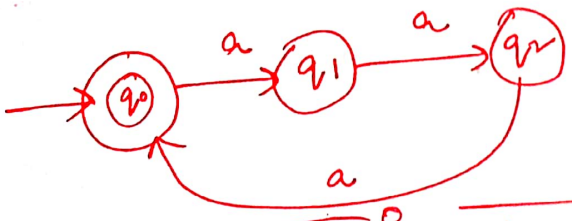


(7) $L = \{a^n \mid n \bmod 3 = 0\}$

$= \{\epsilon, aaa, aaaaaa, \cdots\}$



Not minimal



(8) $L = \{a^{2^n} \mid n \geq 0\}$

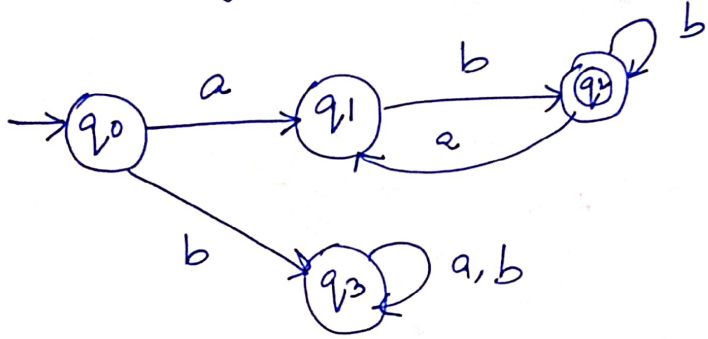
X $\{a, aa, aaaa, aaaaaaaa, \cdots\}$

Not possible.

Examples

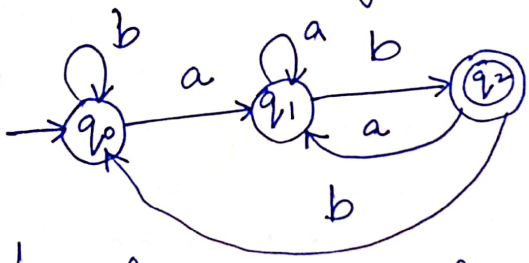
(1) DfA accepts all string of a's and b's

→ each starting with a and ending with b.



$L = \{ab, abbb, aaabb, abab, aabb, \ldots\}$

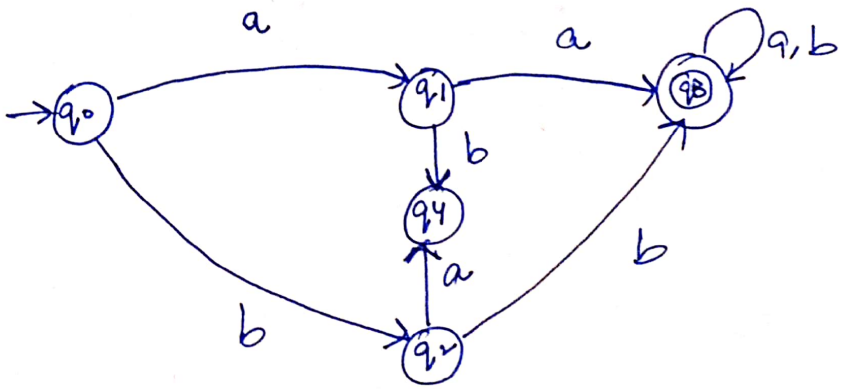
(2) Each string with ab ending.



$L = \{ab, aab, bab, aaab, \ldots\}$

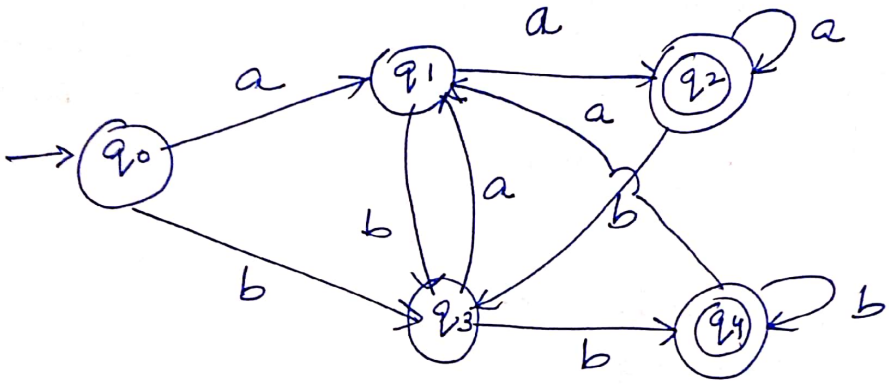
(3) first two symbols are same in every string. aa, bb.

$L = \{aab, bba, aaa, bbb, \ldots\}$



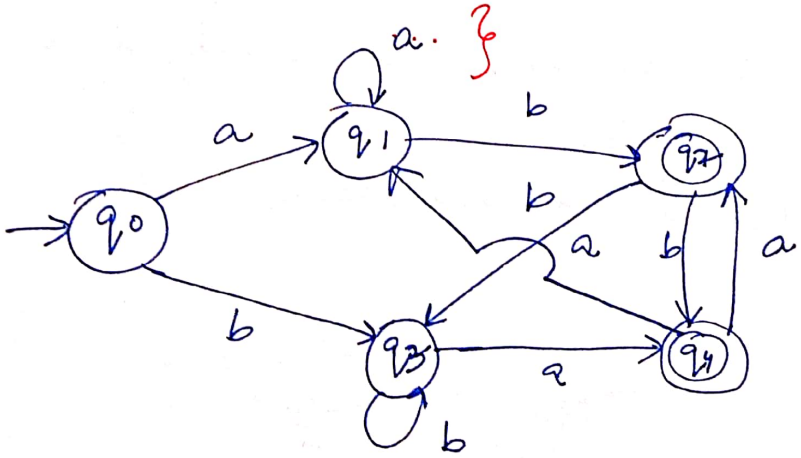
(4) Last two symbols are same in every string. ata

$L = \{aa, bb, baa, abb, \cdots\}$

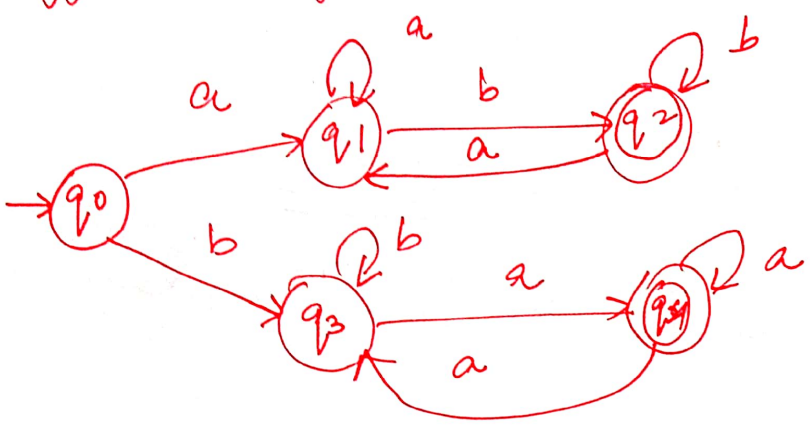


(5) Last two symbols are different.

$L = \{ab, \cancel{ba}, aab, aaab, bbab, bba, \cancel{abb}, bbba, bbbba, aaba, a. \}$



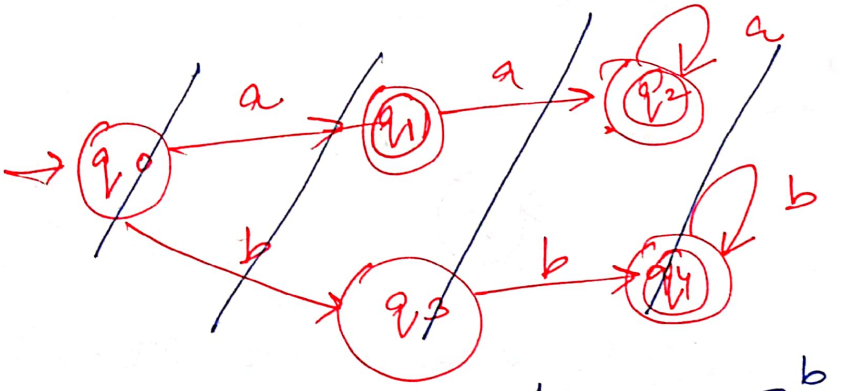
∴ each string starts and ends with different symbol.

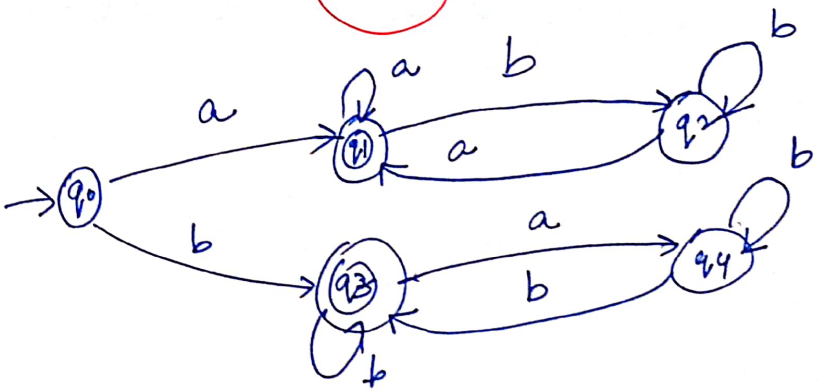


$L = \{ab, ba, aab, baa, abab, baba, aaab, \ldots\}$

(7) Starting and ending with same symbols.

$L = \{aa, aba, a, b, bab, aaba, \ldots\}$





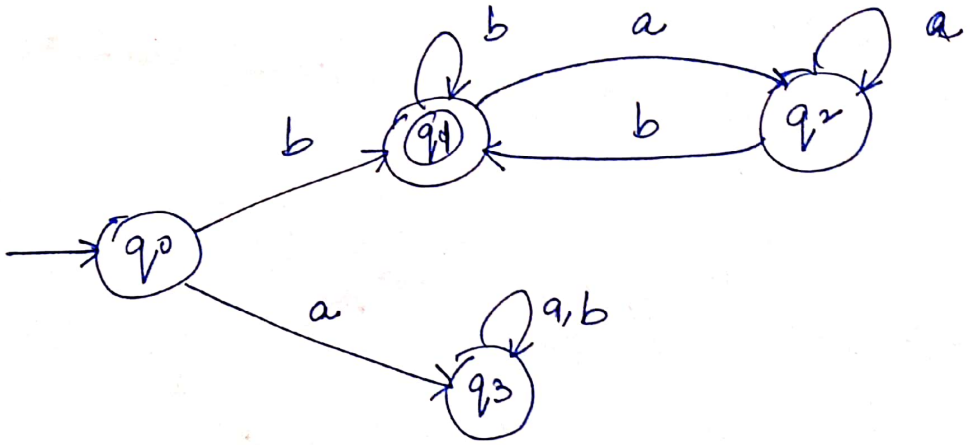
(8)

10 examples:

* $7 + 3 = 10 \Rightarrow$ Mon / Tuesday.

(4) Extra

Identify language accepted by following DFA:



Starts and ends at b.

* NFA → Non-deterministic finite Automata

In NFA, from each and every state on every i/p symbols there may be 0 transition or one transition or more than one transition may exists.

$(Q, \Sigma, \delta, q_0, F)$

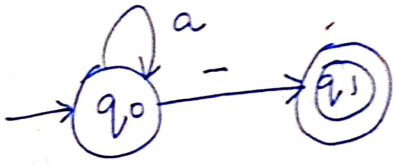
$Q \rightarrow$ finite no. of states
$\Sigma \rightarrow$ i/p alphabets
$\delta \rightarrow$ Transition function
$q_0 \rightarrow$ Initial state
$F \rightarrow$ set of final state

$\delta: Q \times \Sigma \rightarrow 2^Q$ or $P(Q) \rightarrow$ Power Set

→ Construction for NFA is easier than DFA.
→ Language detection :→ easy.
→ Complementation not possible for NFA.
→ Minimization not possible for NFA.
→ Takes less time to construct

Every DFA is NFA but every NFA need not to be DFA.

Ex:-①



Power Set

$\delta(q_0, a) = \{\}, \{q_0\}, \{q_1\}, \{q_0, q_1\}$

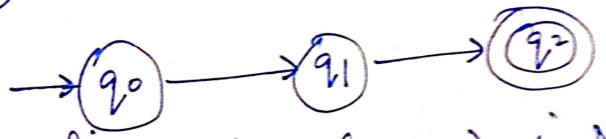
possible next states for a to q_0

2^Q

$2^2 = 4$

2^Q

Ex:-②



possibilities for (q_0, a) in NFA:

$\{\}, \{q_0\}, \{q_1\}, \{q_2\},$
$\{q_0 q_1\}, \{q_0 q_2\}, \{q_1 q_2\}$
$\{q_0 q_1 q_2\}$

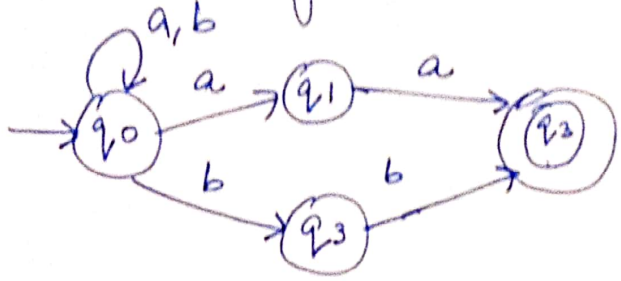
Power Set

2^Q

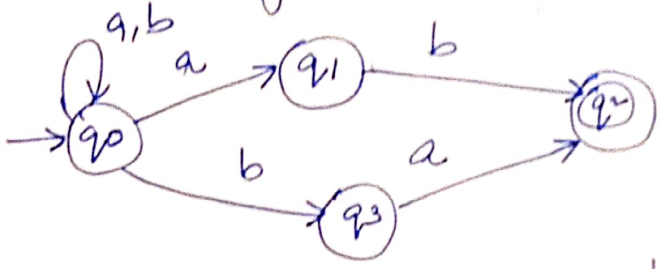
$2^3 = 8$

Ex:-① Construct a NDFA that accept all string of a's and b's where each string ends at abab.

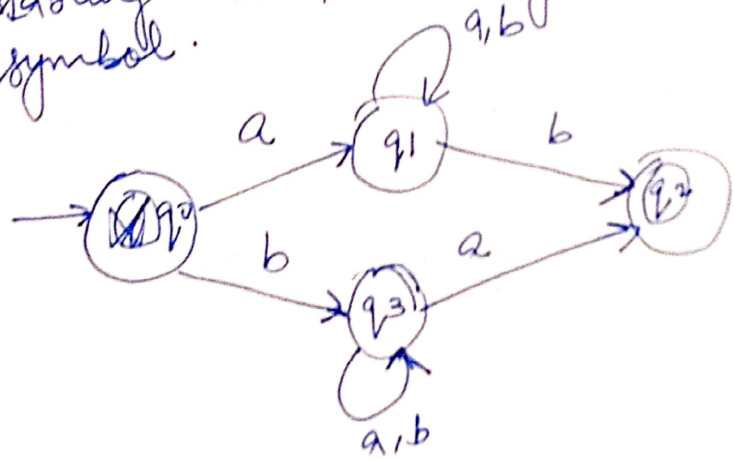
* Last two symbols are same



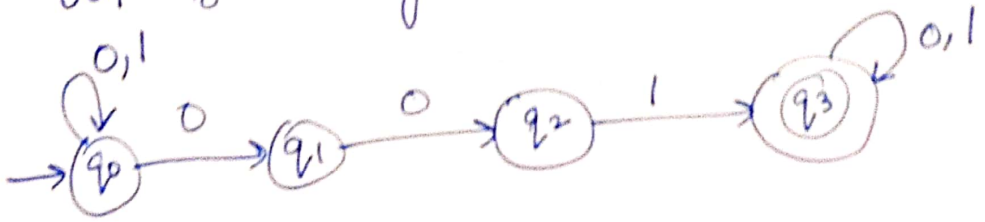
(3) Last two symbols are different



(4) Starting and ending with different symbol.

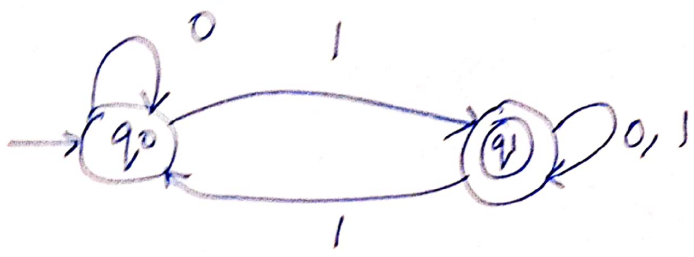


(5) '001' substring



* Conversion of NFA to DFA :-

(1)

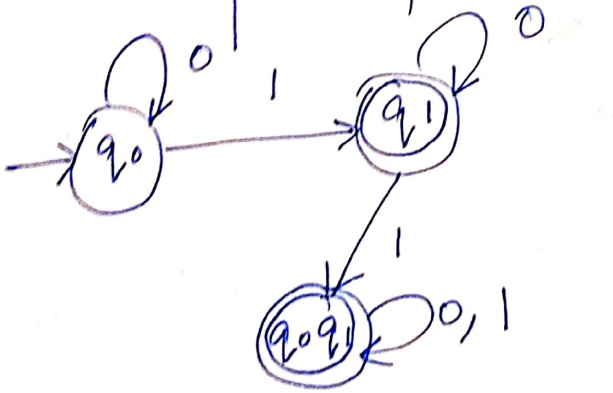


Transition table

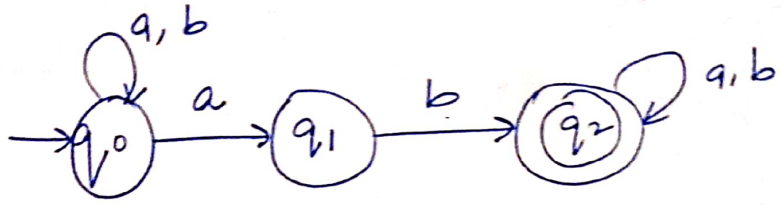
	0	1
→ q_0	q_0	q_1
q_1	q_1	q_0q_1

⇓

	0	1
→ q_0	q_0	q_1
* q_1	q_1	q_0q_1
* q_0q_1	q_0q_1	q_0q_1

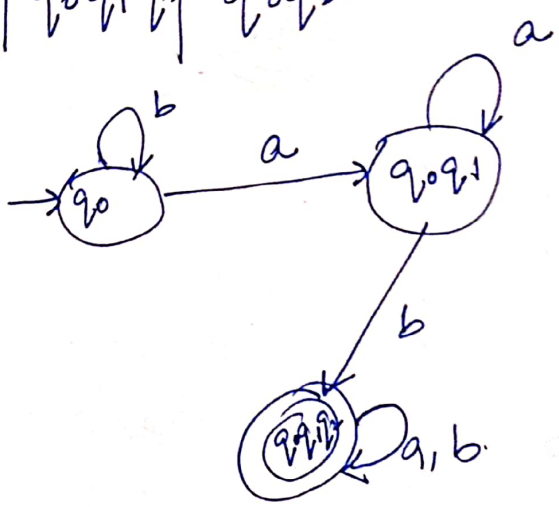


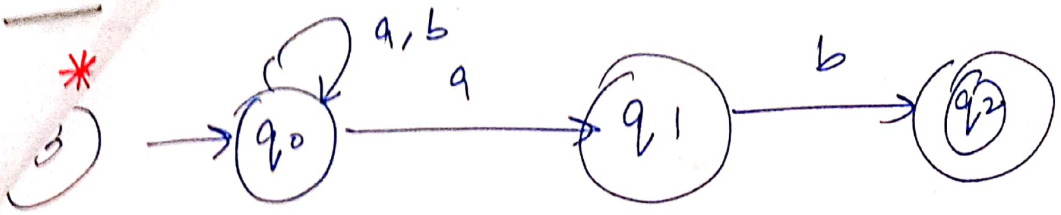
(2)



	a	b
→ q_0	q_0q_1	q_0
q_1	$\emptyset$	q_2 X
* q_2	q_2	q_2

	a	b
→ q_0	q_0q_1	q_0
q_0q_1	q_0q_1	q_0q_2
X q_0q_2	$q_0q_1q_2$	q_0q_2] same
$q_0q_1q_2$	$q_0q_1q_2$	q_0q_2



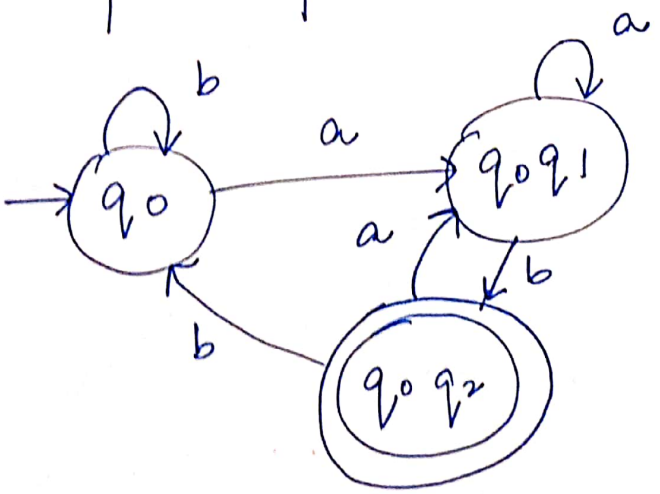


	a	b	
→ q_0	$q_0 q_1$	q_0	
q_1	—	q_2	x
* q_2	—	—	x

Blank → x
eliminate in final

↓

	a	b
→ q_0	$q_0 q_1$	q_0
$q_0 q_1$	$q_0 q_1$	$q_0 q_2$
* $q_0 q_2$	$q_0 q_1$	q_0



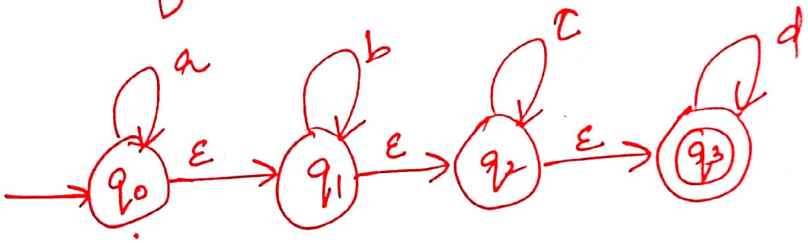
* ε-NfA :-

NfA with ε transition = ε-NfA

$(Q, \Sigma, \delta, q_0, F)$

$\delta: Q \times \Sigma \cup \{\varepsilon\} \rightarrow 2^Q$

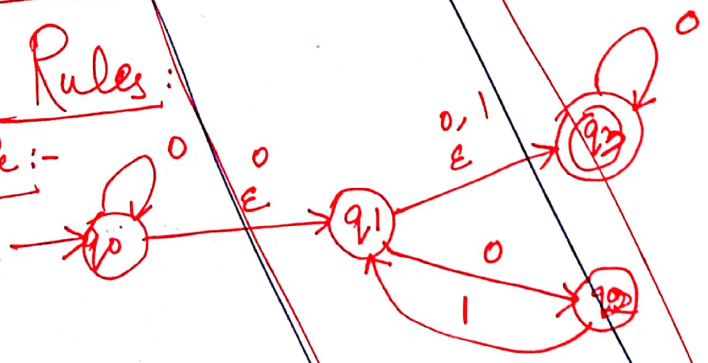
$L = \{a^n b^m c^k d^l \mid l, m, n, k \geq 0\}$



* Convert ε-NfA to NfA :-

Rules:

Example:-



$Q = \{q_0, q_1, q_2, q_3\}$

$\Sigma = \{0, 1\}$

$F = \{q_3\}$

Step 1:- find null closure of all states

State	Set
q_0	$\{q_0, q_1, q_3\}$
q_1	$\{q_1, q_3\}$
q_2	$\{q_2\}$
q_3	$\{q_3\}$

Step 2:-

State	0	1
q_0	$\{q_0, q_1, q_2, q_3\}$	—
q_1	q_2, q_3	—
q_2	—	$\{q_1, q_3\}$
q_3	q_2	—

ε-NFA

Questions

① Convert ε-NFA to NFA:-

Step 1:- find ε-closure of each state:

ε-closure $(q_0) = \{q_0, q_1, q_2\}$

ε-closure $(q_1) = \{q_1, q_2\}$

ε-closure $(q_2) = \{q_2\}$

$q_0 \rightarrow$ ε-closure $\rightarrow$ with null input/no input, we can reach to q_0, q_1, q_2.

	0	1
q_0	$\{q_0, q_1, q_2\}$	

$\delta'(q_0, 0) = \varepsilon$-closure $(\delta(\delta''(q_0, \varepsilon), 0))$

$= \varepsilon$-closure $(\delta(\varepsilon$-closure $(q_0), 0))$

$= \varepsilon$-closure $(\delta(q_0, q_1, q_2), 0))$

$= \varepsilon$-closure $(q_0) = \{q_0, q_1, q_2\}$

$\delta'(q_0,1) = \varepsilon\text{-closure}(\delta(\delta''(q_0,\varepsilon),1))$

$= \varepsilon\text{-closure}(\delta(\varepsilon\text{-closure}(q_0),1))$

$= \varepsilon\text{-closure}(\delta(q_0,q_1,q_2),1))$

$= \varepsilon\text{-closure}(\delta(q_0,1) \cup \delta(q_1,1) \cup \delta(q_2,1))$

$\delta'(q_0,2) = q_2$ $= \varepsilon\text{-closure}(\emptyset \cup q_1 \cup \emptyset) = \{q_1,q_2\}$

$= \varepsilon\text{-closure}(q_1) = \{q_1,q_2\}$

$\delta'(q_1,0) = \varepsilon\text{-closure}(\delta(\delta''(q_1,\varepsilon),0))$

$= \varepsilon\text{-closure}(\delta(\varepsilon\text{-closure}(q_1),0))$

$= \varepsilon\text{-closure}(\delta(q_1,q_2),0))$

$= \varepsilon\text{-closure}(\delta(q_1,0) \cup \delta(q_2,0))$

~~$\delta'(q_1,2)$~~ $= \varepsilon\text{-closure}(\emptyset \cup \emptyset)$

$= \emptyset$.

$\delta'(q_1,1) = \varepsilon\text{-closure}(\delta(\delta''(q_1,\varepsilon),1))$

$= \varepsilon\text{-closure}(\delta(\varepsilon\text{-closure}(q_1),1))$

$= \varepsilon\text{-closure}(\delta(q_1,q_2),1)$

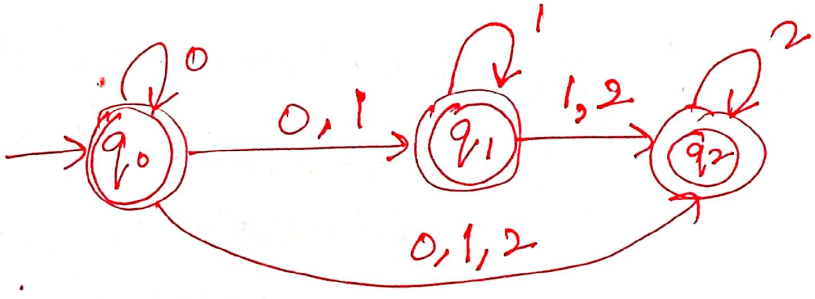
$= \varepsilon\text{-closure}(\delta(q_1,1) \cup \delta(q_2,1)))$

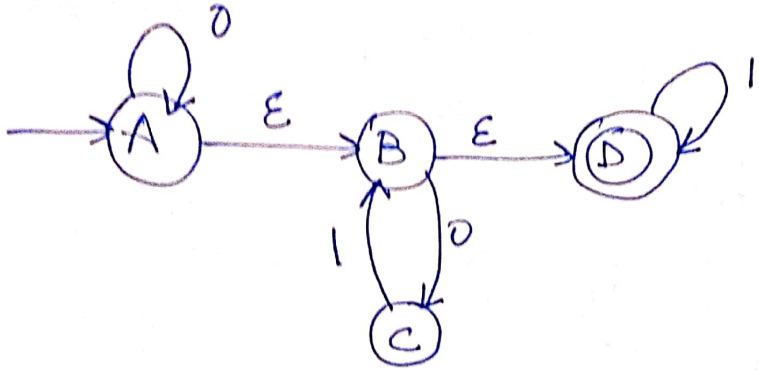
$\delta'(q_1,2) = q_2$ $= \varepsilon\text{-closure}(q_1 \cup \emptyset) = \{q_1,q_2\}$

$\delta'(q_2,0) = \emptyset$

$\delta'(q_2,1) = \emptyset$

$\delta'(q_2,2) = \{q_2\}$





ε-closure (A) = {A, B, D}

ε-closure (B) = {B, D}

ε-closure (C) = {C}

ε-closure (D) = {D}

$\delta'(\phi, 0) = \varepsilon\text{-closure}(\delta(\delta''(\phi, \varepsilon), 0))$

$= \varepsilon\text{-closure}(\delta(\varepsilon\text{-closure}(\phi), 0))$

$= \varepsilon\text{-closure}(\delta(A, B, D), 0))$

$= \varepsilon\text{-closure}(A \cup C \cup \emptyset)$

$= \{A, B, C, D\}$

$\delta'(A, 1) = \varepsilon\text{-closure}(\delta(A, B, D), 1))$

$= \varepsilon\text{-closure}(\emptyset \cup \emptyset \cup D)$

$= \varepsilon\text{-closure}(D) = D$

$\delta'(B, 0) = \varepsilon\text{-closure}(\delta(\delta''(B, \varepsilon), 0))$

$= \varepsilon\text{-closure}(\delta(\varepsilon\text{-closure}(B), 0))$

$= \varepsilon\text{-closure}(\delta(B, D), 0))$

$= \varepsilon\text{-closure}(\emptyset \cup \emptyset)$

$= \varepsilon\text{-closure}(\emptyset) = \emptyset$

$\delta(B,0)$ = ε-closure $(\delta(\delta''(B,\varepsilon),0))$
= ε-closure $(\delta(\varepsilon\text{-closure}(B),0))$
= ε-closure $(\delta(B,D),0))$
= ε-closure $(\delta(B,0) \cup \delta(D,0))$
= ε-closure $(C \cup \phi)$
= ε-closure (C)
= $\{C\}$

$\delta(B,1)$ = ε-closure $(\delta(\delta''(B,\varepsilon),1))$
= ε-closure $(\delta(\varepsilon\text{-closure}(B),1))$
= ε-closure $(\delta(B,D),1))$
= ε-closure $(\delta(B,1) \cup \delta(D,1))$
= ε-closure $(\phi \cup D)$
= ε-closure $(D) = D$

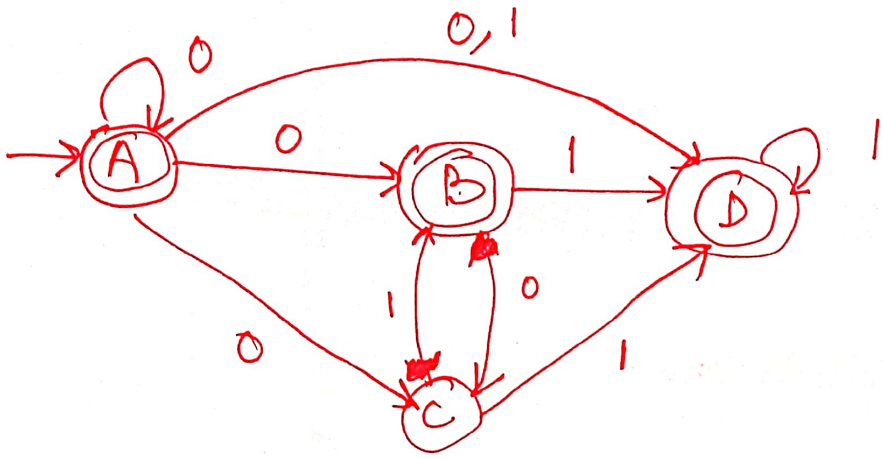
$\delta(C,0)$ = ε-closure $(\delta(\delta''(C,\varepsilon),0))$
= ε-closure $(\delta(\varepsilon\text{-closure}(C),0))$
= ε-closure $(\delta(C,0))$
= ϕ

$\delta(C,1)$ = ε-closure $(\delta(C,1))$ = ε-closure (B)
$= \{B,D\}$

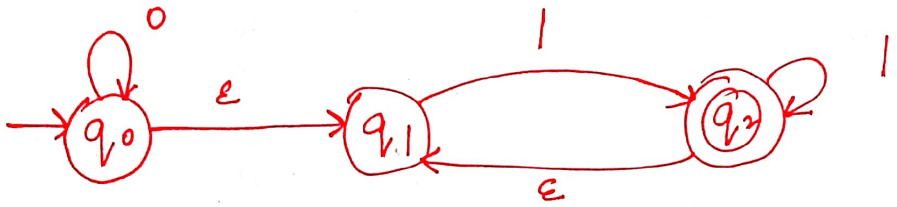
$\delta(D,0)$ = ε-closure $(\delta(\delta''(D,\varepsilon),0))$
= ε-closure $(\delta(\varepsilon\text{-closure}(D),0))$
= ε-closure $(\delta(D,0))$ = ϕ

$\delta(D,1)$ = ε-closure $(\delta(D,1))$ = ε-closure (D)
= D.

(a)



(b)



ε-closure $(q_0) = \{q_0, q_1, q_2\}$

ε-closure $(q_1) = \{q_1, q_2\}$

ε-closure $(q_2) = \{q_2, q_1\}$

$\delta(q_0, 0) = \varepsilon$-closure $(\delta(\delta''(q_0, \varepsilon), 0)) = \varepsilon$-closure $(\delta(\varepsilon\text{-closure}(q_0), 0))$

$= \varepsilon$-closure $(\delta(q_0, q_1, q_2), 0))$

$= \varepsilon$-closure $(q_0 \cup \phi \cup \phi) = \{q_0, q_1, q_2\}$ ✓

$\delta(q_0, 1) = \varepsilon$-closure $(\delta(q_0, q_1, q_2), 1))$

$= \varepsilon$-closure $(\phi \cup q_2 \cup q_2)$

$= \{q_2\}$

$\delta(q_1, 0) = \varepsilon$-closure $(\delta(\varepsilon$-closure $(q_1), 0))$

$= \varepsilon$-closure $(\delta(q_1, q_2), 0))$

$= \varepsilon$-closure $(\delta(q_1, 0) \cup \delta(q_2, 0)))$

$= \varepsilon$-closure $(\phi \cup \phi) = \phi$.

$\delta(q_1, 1) = \varepsilon$-closure $(\delta(q_1, q_2), 1))$

$= \varepsilon$-closure $(q_2 \cup q_2)$

$= \{q_2\}$.

$\delta(q_2, 0) = \varepsilon\text{-closure}(\delta(\varepsilon\text{-closure}(q_2), 0))$

$= \varepsilon\text{-closure}(\delta(q_2, 0))$

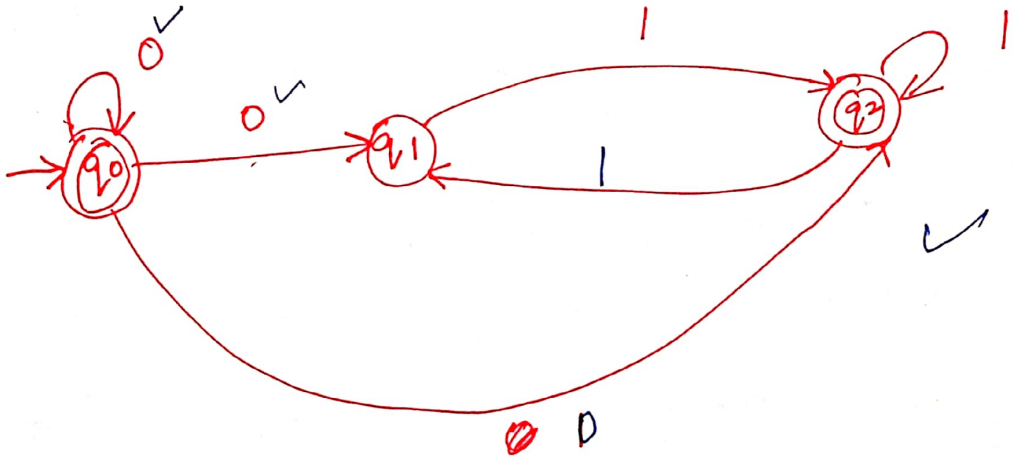
$= \varepsilon\text{-closure}(\phi)$

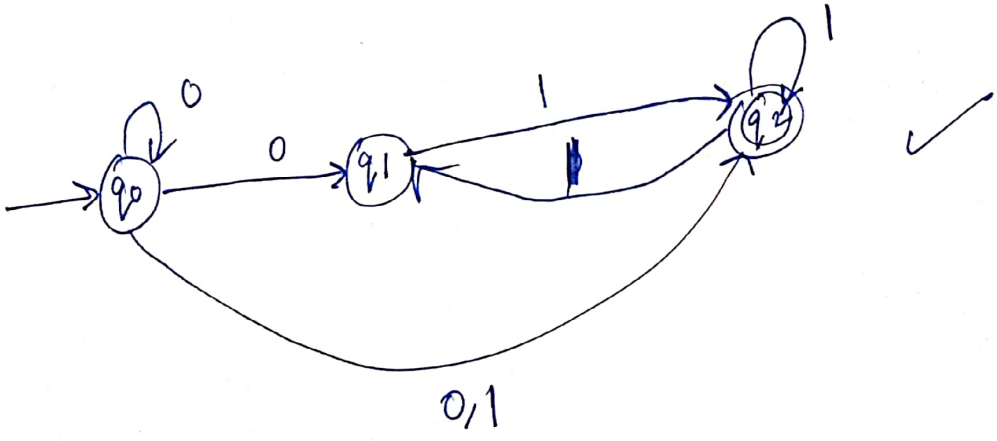
$= \phi$

$\delta(q_2, 1) = \varepsilon\text{-closure}(\delta(q_2, 1))$

$= \varepsilon\text{-closure}(q_2)$

$= \{q_2, q_1\}$





28/08/2023

(1)

*Where do we use?

DfA, NfA and ε-NfA

Language recognizers

(a) DfA :→ Traffic Sensitive lights / Traffic lights, Elevators, etc.

(b) NFA :→ Games
↓
input cannot be controlled.
↓
Throw a coin with same motion and strengths
↓
Not necessarily same result (Heads or Tails)

Ludo, Playing cards, Gun shot (wind), etc.

ATM Machine

User → Insert Card → Money ✓
→ Fail Pin
→ Enough money ×

(c) ε-NfA :→ Change state without input.

Car → 0
Start → to 1
Rest → 0

Gear 1
Clutch

Rest —ε→ Start

Sometimes
Rest 1 + clutch.
Start 1

② Mealy and Moore Machines

→ O/P Generator / FA with O/P

→ Not language recognizer

No final states

→ Both machines are deterministic in nature. Hence, on each and every state on every input symbol exactly one transition exists.

→ Circuit designing

* Mealy and Moore are formally defined as :-

$(Q, \Sigma, q_0, \delta, \Delta, \lambda)$

Q → finite no. of states

Σ → input symbol

q_0 → initial state

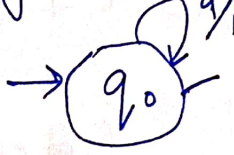
δ → Transition

$Q \times \Sigma \rightarrow Q$

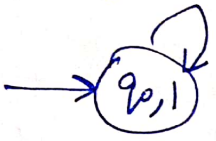
Δ → o/p alphabet

λ → o/p function

Mealy: $\lambda : Q \times \Sigma \rightarrow \Delta$

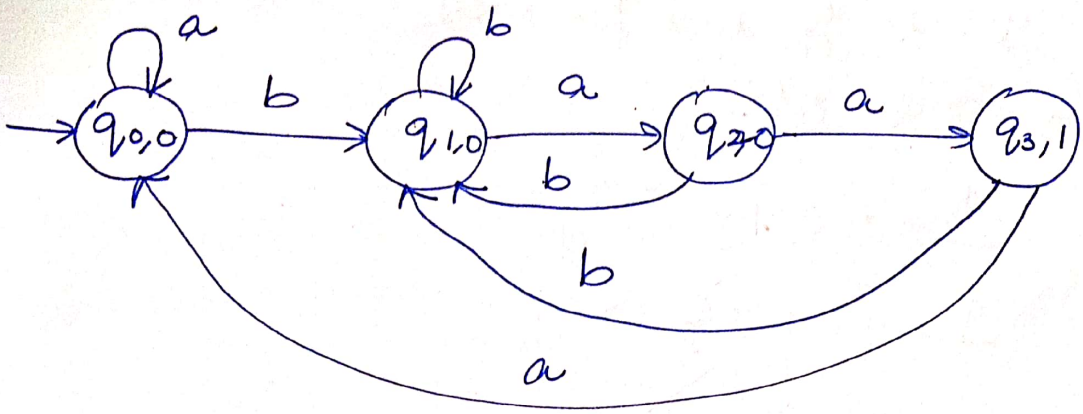


Moore: $\lambda : Q \rightarrow \Delta$



Construct Moore Machine

bbaa → (1) count

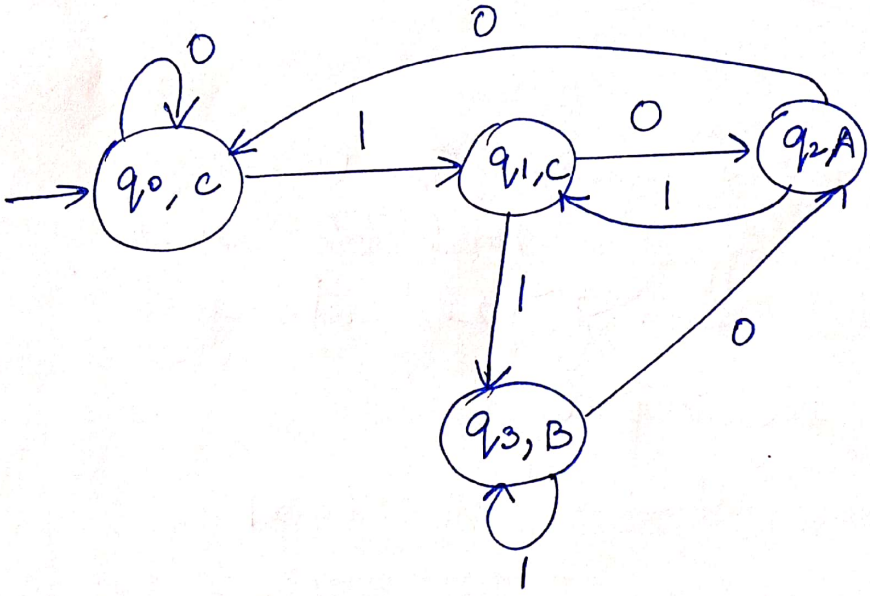


Q4:-

A → 10

B → 11

C → otherwise



End

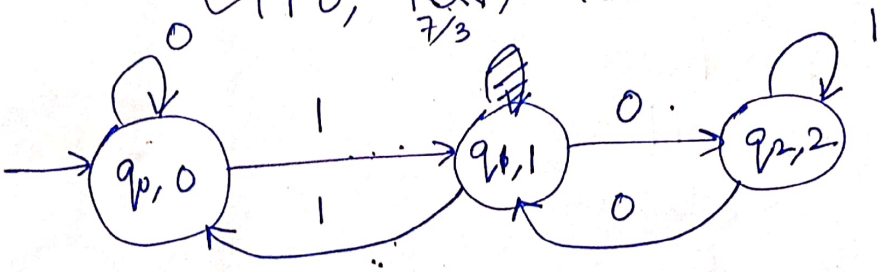
Moore Machine

Q1:- Construct Moore machine that takes all binary no. as i/p and produces remainder/residue modulo 3 as o/p.

$\Sigma = \{0, 1\}$ → i/p

$\Delta = \{0, 1, 2\}$ → o/p

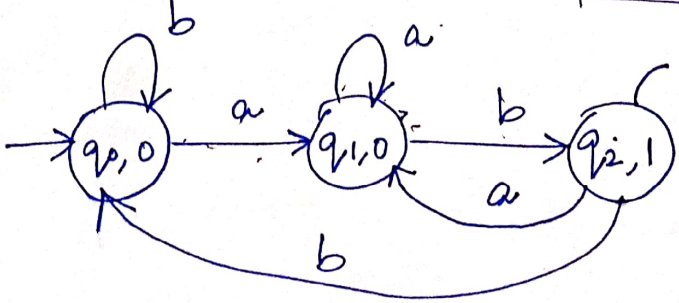
000, 001, 010
011, 100, 101
110, 111, 7/3



Q2:- Construct Moore machine that takes all strings of a's and b's and counts total no. of 'ab' strings present in the input.

ab → m/c → 1

abab → m/c → 2

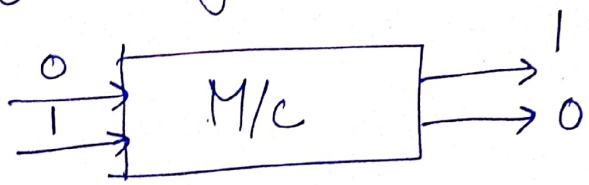


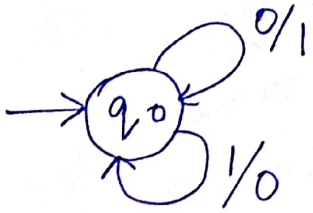
9/10/2023

Mealy → o/p is associated with transition

* Moore → o/p is associated with state.

Q1:- Construct a Mealy machine that represent behavior of 1's complement circuit of binary number.





Q2:- Construct a Mealy machine that represent the behavior of binary incrementor. (it increments no. by 1)

We read string from LSB to MSB and carry is discarded

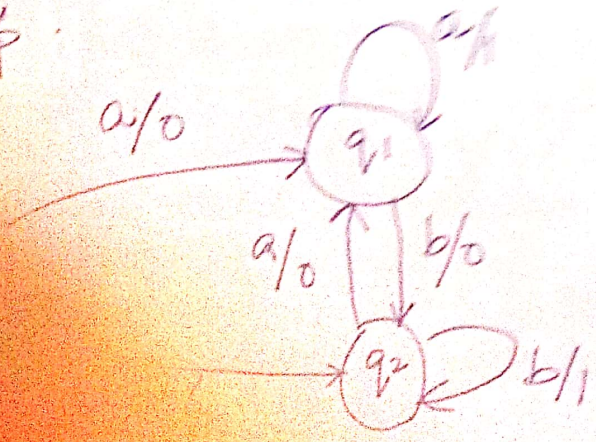
8 + 1 = 9

i/p → 1 00 0
o/p → 1 00 1

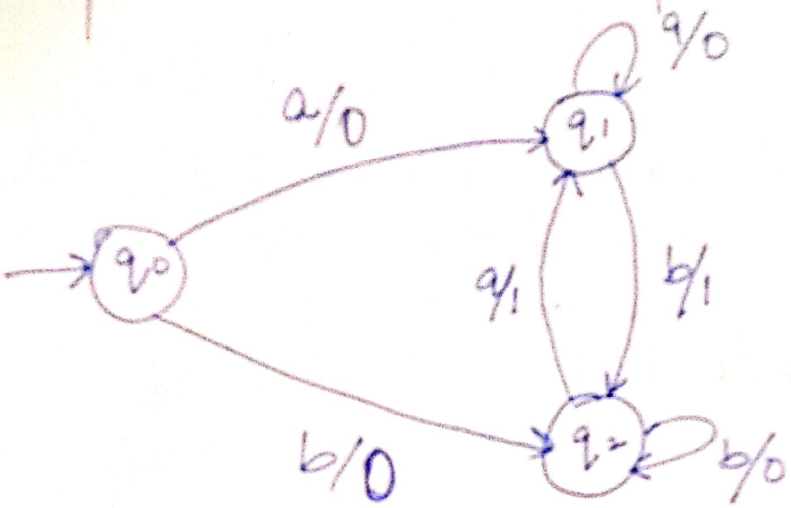
i/p → 0000
o/p → 0001 → 00 10

LSB → read → replace 1's with 0's first 0 with 1

Q3:- Construct moore machine that takes all strings of 0's and 1's and produces 1 as o/p - if last two symbols are same, otherwise produces 0 as o/p.

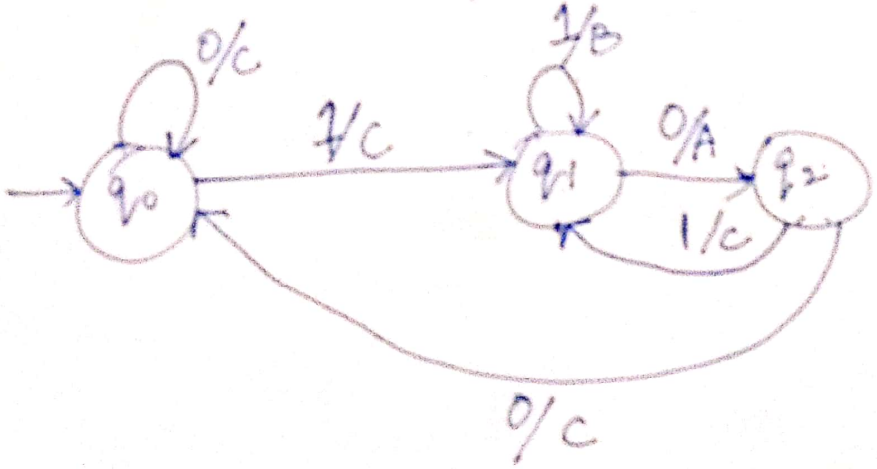


last two symbols are different produces '1' as output, otherwise produces '0' as output. / same o/p other 1



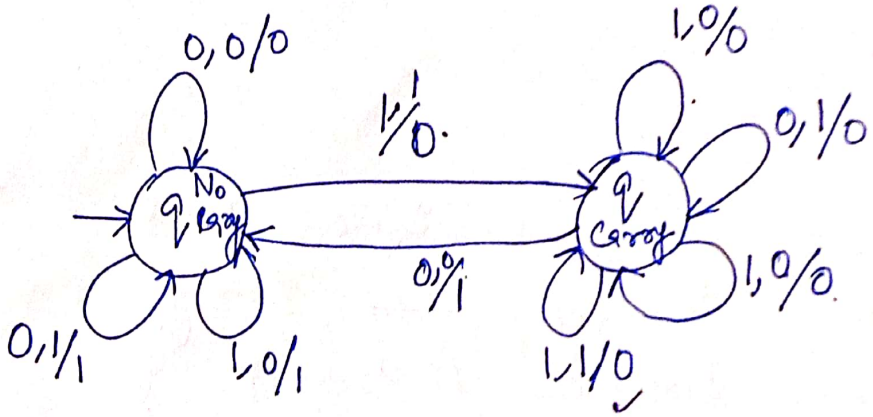
Q5:- Construct Mealy machine that takes all binary ~~non~~ strings as i/p and produces A as o/p if the i/p is ending as 10, producing B as o/p if i/p is ending as 11, otherwise C as o/p.

10 → A
11 → B
→ C



Q6:- Construct Mealy Machine that represents the behavior of binary adder.

1	0	1	1	0	1	1	0	0
0	1	0	1	0	1	0	1	0
1	0	0	0	0	1	0	1	0
			Carry				No carry	



* Decision properties of FA

Decidable problem

↓

if there exists an algorithm to solve the problem.

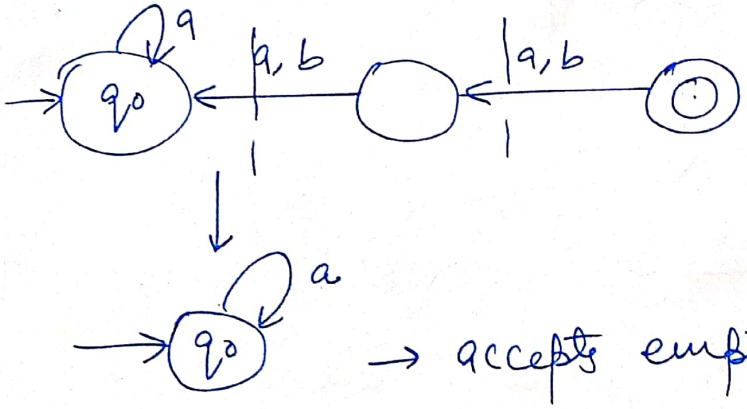
Example:

1) Emptyness problem
2) Finiteness problem
3) Equivalence problem
4) Membership problem

1) Emptyness problem

↓

Where FA accepting empty language or non-empty language.



→ accepts empty language.

Algo:-

→ Remove inaccessible state

→ If in the resulting automata, there exist atleast one final state, then automata accept non-empty otherwise empty.

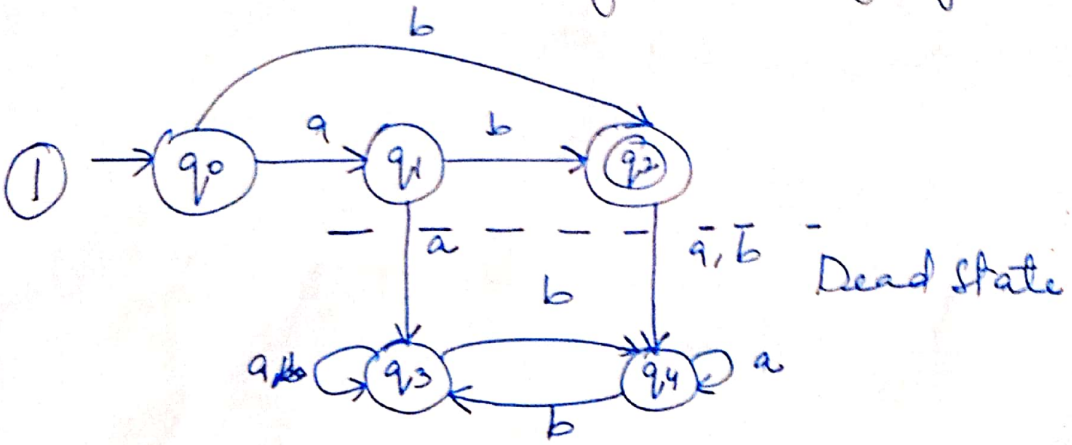
2) Finiteness problem

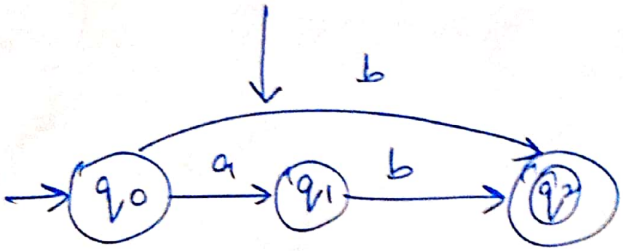
↓

language accepted by FA is finite or infinite.

Algo:-

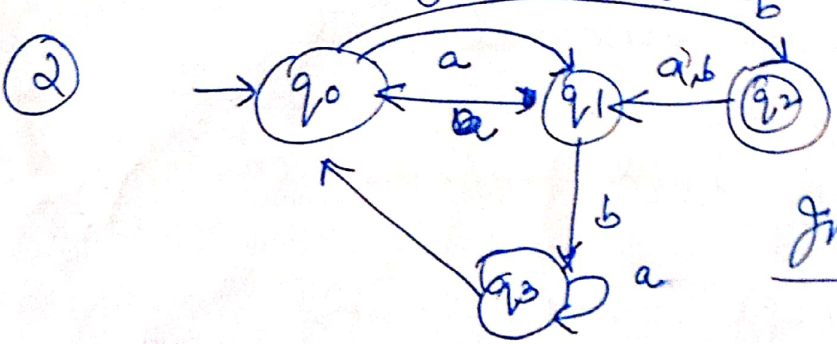
- Remove all inaccessible states
- Dead state
- loop/cycle :→ Infinite language
  Otherwise finite language.





ab or b (No loop)

finite language



loop ✓

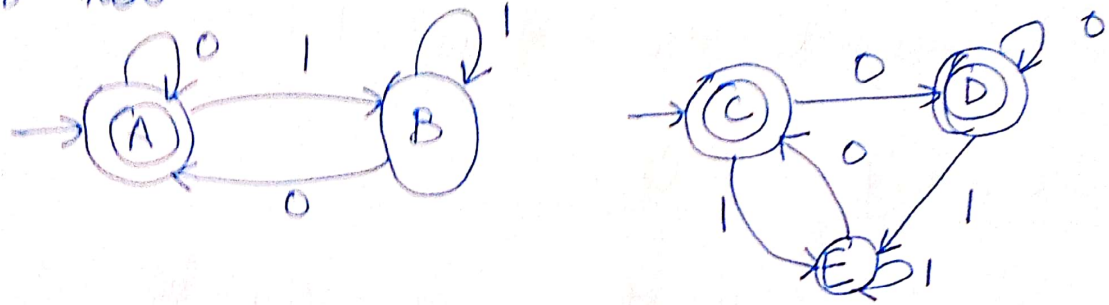
Infinite language

Equivalence

Checking whether language accepted by given two FA are equal or not.

Comparison table algorithm exist to check given two automata are equal or not.

(1)

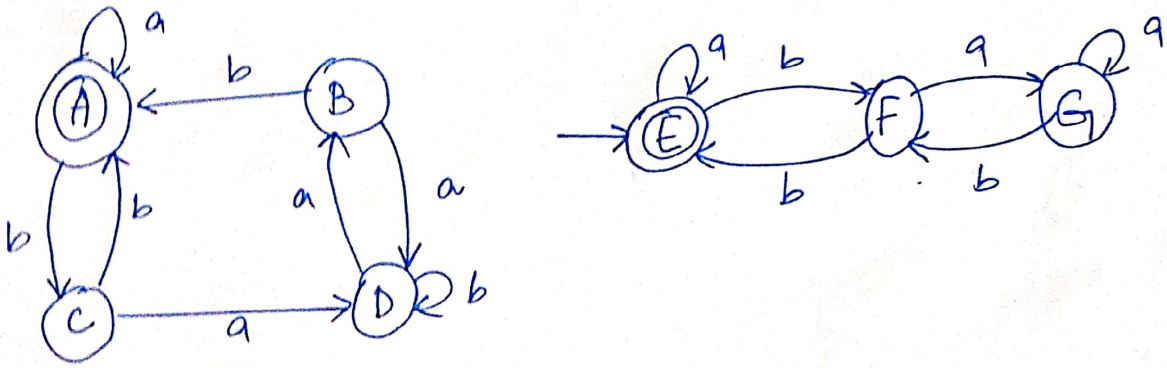


	0	1
A·C	A·D	B·E
A·D	A·D	B·E
B·E	A·C	B·E

There is no pair as (final, Non-final) or (Non-final, final).

Hence both are equal.

(2)



	a	b
A·E	A·E	C·F
C·f	D·G	A·E
D·G	B·G	D·F
B·G	D·G	A·F

(final, Non-final).

Both automata are not equal.

4> Membership Problem

Checking whether given string is a member of given automata or not.

String acceptance method is used.

abaa → accepted by fA
✓ member

if not → not member